

José Antonio Vacas Martínez

Aprende a programar Python



Logo Python

Versión 0.99.2



Licencia CC by SA

by @javacasm

José Antonio Vacas Martínez

<https://elCacharreo.com>

Octubre 2022

Colecciones

Una colección es una estructura que como su nombre indica nos permite almacenar varios elementos. Existen diferentes tipos de colecciones, cada una con diferentes características lo que lo hace más indicado para según qué tipo de almacenamiento.

El tipo más sencillo son las **Tuplas** (tuple), un conjunto invariable de elementos, no tienen que ser del mismo tipo.

Veremos también las **Listas** (list) nos permiten guardar también múltiples objetos y de cualquier tipo, con la peculiaridad de que están ordenados. Disponen de todos los mecanismos para trabajar dinámicamente con ellas, añadiendo o quitando elementos, reordenando.

Otro tipo muy importante son los **Diccionarios** (dict), donde tenemos un conjunto de pares que no están ordenados, formados por una clave (key) y un valor (value). La clave tiene que ser única.

También existen los **Conjuntos** (sets), que almacenan valores sin ordenar y que no pueden estar repetidos. Sólo se pueden guardar elementos inmutables (que no cambian) y que de los que se puede obtener un valor **hash** (algo similar a un id único)

Para cada caso escogeremos la estructura que nos facilite más la programación, donde la propia naturaleza de la estructuras nos será muy útil.

Por ejemplo para guardar una lista de propiedades, parece más interesante utilizar un diccionario, donde guardaremos de forma natural el nombre de la propiedad y su valor.

Si necesitamos mantener varios elementos sin que se produzca repetición, parece que un conjunto sería lo más indicado.

Tuplas

El tipo más sencillo de colección son las **Tuplas**, que son un conjunto invariable de elementos, es decir, una vez creadas no podemos modificarlas, ni añadir ni quitar elementos. Los elementos no tienen que ser del mismo tipo, ni tampoco podemos cambiarlos de valor, es decir sus elementos también tienen que ser invariantes, por lo que no pueden ser variables.

¿Qué datos podemos usar que no necesiten cambio? Por ejemplo los días de la semana:

```
>>> dias = ('L', 'M', 'X', 'J', 'V', 'S', 'D')
```

No hay ningún problema en que se repitan elementos dentro de la tupla.

Las definimos poniendo los elementos que la forman entre paréntesis:

```
>>> coordenadas = (1,3,4)
>>> elementos = ('a',7,'hola qué tal')
```

Podemos acceder a un elemento de la tupla con su índice que irá entre 0 y N-1 siendo N el número de elementos de la tupla, indicándolo entre llaves:

```
>>> elementos[0]
'a'
```

Si intentamos acceder a un elemento más allá del último obtendremos una excepción del tipo *IndexError*.

También podemos acceder a los elementos con índices negativos, en cuyo caso se entiende que índice -1 corresponde al último y accederemos hacia atrás:

```
>>> elementos[-1]
'hola qué tal'
>>> elementos[-2]
7
```

Podemos *sumar* tuplas usando el operador "+" y obtendremos una nueva con los elementos de las 2 sumadas.

Si usamos el operador multiplicación "*" lo que haremos será repetirla N veces.

También podemos acceder a una parte de la tupla usando el índice de elemento inicial y del final separados por ":", obtendremos una lista con los elementos seleccionados:

```
>>> elementos[1:2]
(7,'hola qué tal')
```

También podemos omitir el número inicial o el final si queremos que empiece en el principio o acabe al final de la tupla:

```
>>> festivos = dias[5:]
>>> laborables = dias[:5]
```

Se suele llamar rodaja (slice) a esta parte que tomamos de una colección mayor.

También podemos usar un tercer entero para indicar cada cuantos elementos queremos iterar.

¿Te recuerda a algo? Sí: la función **range** es un generador de tuplas de números.

Iterando

Podemos recorrer los elementos de una tupla usando el bucle for:

```
valores = (1,5,2,7)
for valor in valores:
    print(valor)
```

Podemos conocer el tamaño de una tupla usando el método *len*, lo que nos permite recorrer la tupla usando un bucle *while*

```
valores = (1,5,2,7)
contador = 0
while contador < len(valores):
    print(valores[contador])
    contador += 1
```

Podemos saber si un elemento está en una tupla usando la palabra reservada *in*:

```
>>> 4 in valores
False
```

Que nos dará True o False según si el elemento está o no en la tupla.

```
>>> 'X' in festivos
False
```

También podemos saber cuántas veces aparece un elemento con el método *count()*:

```
>>> valores.count(5)
1
```

Listas

Una lista es una colección en la que los elementos que pueden ser de distintos tipos, están ordenados.

Características de las listas:

- No todos los elementos tienen que ser del mismo tipo.
- La lista es dinámica y puede cambiar de tamaño, es decir, podemos añadir y quitar elementos.
- Los elementos están ordenados, en cuanto a su posición en la lista. No es que por defecto se les aplique un orden al insertarlos. Además dado que pueden ser de distintos tipos, no siempre se puede establecer un orden según su valor.

Si has programado en otro lenguaje, las listas te pueden recordar a los arrays, pero en Python son mucho más dinámicas y versátiles.

Para declararla lo haremos de la siguiente forma:

```
mi_lista = [ 10, "hola", "Pepe",25]
```

Accederemos a cada elemento usando su posición, empezando en 0 y terminando en el elemento N-1, siendo N el número de elementos de la lista.

```
print(mi_lista[2])
```

Podemos modificar el elemento que está en una posición dada:

```
mi_lista = [ 10, "hola", "Pepe",25]
```

```
print("inicialmente",mi_lista[2])
```

```
mi_lista[2] = "Juan"
```

```
print("ahora es",mi_lista[2])
```

También podemos usar un índice negativo siendo **-1** el elemento de la última posición y así hasta llegar al primero.

Si intentamos acceder a un elemento más allá de la última o de la primera obtendremos una excepción **IndexError**

Podemos crear una lista vacía con `lista = []`

SubListas

También podemos recuperar varios elementos de una lista, es lo que se llama una **sublista**. Para ello indicaremos un rango de valores, que son 2 valores numéricos separados por ":", de la siguiente forma **principio:final**, siendo **principio** el índice del primer elemento de la sublista y **final** el índice posterior al último que vamos a recuperar.

```
>>> print(mi_lista[0:2])  
[10, 'hola']
```

Si omitimos el primer valor se entenderá que nos referimos al primer valor de la lista

```
>>> print(mi_lista[:2])  
[10, 'hola']
```

y si omitimos el segundo que queremos llegar hasta el elemento final

```
>>> print(mi_lista[2:])  
['Juan', 25]
```

Si omitimos los dos tendremos la lista entera

```
>>> print(mi_lista[:])  
[10, 'hola', 'Juan', 25]
```

También podemos incluir un tercer número en la sublista (separado por otro :) que indicará cada cuantos elementos contamos al elegirlos (lo que sería el "paso"). Por defecto este número es 1, pero podría ser otro.

```
>>> print(mi_lista[::2])  
[10, 'Juan']
```

Una utilidad de este tercer parámetro es invertir el orden de nuestra lista usando **-1**. Por ejemplo:

```
>>> print(mi_lista[::-1])  
[25, 'Juan', 'hola', 10]
```

Operaciones sobre listas

Si nuestra lista está formada por elementos homogéneos (todo números o todo texto) podremos realizar una serie de operaciones sobre ellas:

- **Ordenación:** `mi_lista.sort()` o `mi_lista.sort(reverse=True)` para invertirla en orden inverso. Una vez ordenada sabremos que el primero o el último serán el mínimo o el máximo valor. Conviene recalcar que la operación se realiza sobre la propia lista, no es que la llamada nos devuelve una lista ordenada.
- Obtener el máximo aplicando la función **max** `max(mi_lista)`.
- Obtener el mínimo aplicando la función **min** `min(mi_lista)`.
- Calcular su longitud con **len** `len(mi_lista)`.

Búsquedas

Una funcionalidad que usaremos muchos son las búsquedas, que podremos hacer con la palabra reservada **in**. Obtendremos como resultado un booleano, siendo *True* que se ha encontrado y *False* en caso contrario.

Conviene destacar que se distingue entre mayúsculas y minúsculas.

```
>>> mi_lista = ['Juan', 'Dionisio', 'Pepe', 'Manolo']  
>>> 'Juan' in mi_lista  
True
```

Una vez que sabemos que el elemento está en la lista podemos preguntar en qué posición con **index**:

```
>>> posicion = mi_lista.index("Juan")  
0
```

Si intentamos buscar un elemento que no está se produce una excepción de tipo **ValueError**.

También podemos contar el número de veces que se repite un valor con **count** :

```
>>> mi_lista.count("Juan")
1
>>> mi_lista.count("juan")
0
```

Como vemos podemos usar la función *count* para ver si un elemento está o no en una lista.

Edición en listas

Hemos visto cómo cambiar, buscar, contar o acceder a elementos de una lista. Pero se pueden hacer más operaciones como las siguientes, a las que llamaremos a partir de nuestra lista original

- **append(elemento)**: Añade un elemento al final de la lista.
- **clear()**: borra todos los elementos de una lista.
- **insert(posicion, valor)**: inserta en *posicion* el valor indicado, desplazando los siguientes.
- **remove(elemento)**: elimina **elemento** de la lista. Para esto mismo podemos usar el "arcaísmo" **del**:

```
del lista[4]
```

- **copy()**: devuelve una copia de la lista. Podemos usarlo para crear una nueva si no queremos alterar la original.
- **reverse(lista)**: invierte el orden de la lista.
- **pop()**: Devuelve el último elemento de la lista y lo retira de la lista. También podemos usarlo para eliminar un elemento indicando el índice a sacar de la lista.

```
lista.pop(2)
```

Esta función nos muestra que podemos usar una lista como soporte para un funcionamiento más complejo como sería una cola o pila, donde vamos eliminando el último elemento.

- **extend(lista2)**: añade a la lista actual el contenido de lista2. Es equivalente a hacer una suma de las dos listas:

```
python lista1.extend(lista2) lista1 = lista1 + lista2
```

- **sort()**: Ordena la lista. Si tenemos una lista que mezcla distintos tipos, por ejemplo números y letras nos dará una excepción de tipo **TypeError**.

También podemos ordenar una lista con la función **sorted**:

```
sorted(list)
```

Recorrer una lista: enumerar

Uno de los usos más frecuentes de las listas es recorrer sus elementos. Ya dijimos que los bucles *for* están pensado para recorrer colecciones. Veamos cómo hacerlo con una lista:

```
amigos = ['Juan', 'Dionisio', 'Pepe', 'Manolo']
for amigo in amigos:
    print(f'Hola {amigo}')
```

A este proceso de recorrer una lista se le denomina enumerar o iterar. A veces necesitamos además de iterar el indicar la posición del elemento. Podemos hacerlo con la palabra reservada **enumerate**, obteniendo tanto el elemento como su posición:

```
amigos = ['Juan', 'Dionisio', 'Pepe', 'Manolo']
for posicion, amigo in enumerate(amigos):
    print(posicion, amigo)
```

Este ejemplo nos muestra que podemos utilizar varias variables en un mismo bucle *for*, algo que haremos con frecuencia.

Creación dinámica de listas

En ocasiones necesitamos generar una lista a partir de otra, realizando una operación sobre sus elementos. Podemos hacerlo con lo que ya sabemos, recorriendo nuestra lista y añadiendo los elementos a una nueva lista:

```
pi = 3.14
radios = [2.0, 35, 7.1]
circunferencia = [] # Creamos la lista vacía
for radio in radios:
    circunferencia.append(radio*pi)
```

Pero como ésto es algo muy frecuente, Python incorpora una manera muy cómoda de hacerlo, donde especificamos dentro de la lista la manera en la que calculamos cada elemento a partir de un bucle *for*:

```
pi = 3.14
radios = [2.0, 35, 7.1]
circunferencias = [pi * num for num in radios]
```


Matrices

Hemos dicho que una lista puede contener cualquier cosa. De hecho podemos hacer una lista que contenga listas, es lo que llamamos una matriz. Veamos un ejemplo

```
fila1 = [1, 3, 5]
fila2 = [9, 3, 5]
fila3 = [7, 3, 8]

matriz = [fila1, fila2, fila3]

print(matriz)

print(matriz[1][2])
```

Como vemos necesitamos 2 índices para acceder a un elemento dado.

Diccionarios

Un diccionario es una colección de pares etiqueta/valor, separados por el signo ":". También se usa la palabra clave o llave para referirse a la etiqueta.

Se usan las llaves, expresados de la siguiente forma (usaremos el que parece ser el ejemplo por excelencia de los diccionarios: los lenguajes de programación y su fecha de creación):

```
>>> lenguajes = { 'C':1972, 'python':1991, 'Java':1996}
>>> print(lenguajes)
{'C': 1972, 'python': 1991, 'Java': 1996}
```

Podemos crear un diccionario vacío con:

```
diccionario = {}
```

Para recuperar un valor preguntaremos por una llave:

```
>>> print(lenguajes['C'])
1972
```

Si el elemento no existe se produce una excepción de tipo **KeyError**.

Podemos evitar el error usando `get('clave')` que devolverá **None** si la clave no está en el diccionario. Podemos pasarle al método `get` un argumento que se devolverá como valor por defecto si la clave no existe:

```
>>> print(lenguajes.get('C++', 'Año desconocido'))
'Año desconocido'
```

Si queremos obtener un elemento y a la vez retirarlo del diccionario usaremos la función **pop()**.

Otra forma de ver si una clave está en el diccionario es usando **in**:

```
>>> 'BASIC' in lenguajes
False
```

Asignando un nuevo elemento se incluirá en el diccionario:

```
>>> lenguajes['C++'] = 1983
```

Podemos borrar un elemento usando la función **del**:

```
>>> del lenguajes['Java']
```

Podemos crear diccionarios con esta otra sintaxis:

```
>>> lenguajes = dict(C=1972, Java=1996, Python=1991)
>>> print(lenguajes)
{'C': 1972, 'Java': 1996, 'Python': 1991}
```

O a partir de listas, asignando a todos el mismo valor:

```
>>> oldLenguajes = ['Assembler', 'Lisp', 'Pascal']
lenguajesAntiguos = dict.fromkeys(oldLenguajes, 0)
>>> print(lenguajesAntiguos)
{'Assembler': 0, 'Lisp': 0, 'Pascal': 0}
```

Iterando en los diccionarios

Podemos iterar un diccionario usando las claves:

```
>>> for clave in lenguajes:
...     print(clave)
C
Java
Python
```

También podemos iterar por los valores usando la función **values()** (Cuidado con olvidar los paréntesis de **values()**, que es una función):

```
>>> for valor in lenguajes.values():
...     print(valor)
1972
1996
1991
```

Del mismo modo que hemos accedido a todos los valores almacenados en el diccionario, también podemos acceder a todas las claves usando la función **keys()**:

```
>>> for key in lenguajes.keys():
...     print(key)
C
Java
Python

>>> 'C++' in lenguajes.keys()

False
```

Aunque no es una lista, podemos usarla como tal.

Incluso podemos iterar por ambos a la vez usando la función *items()* (Cuidado con olvidar los paréntesis de *items()*, que es una función):

```
>>> for clave,valor in lenguajes.items():
...     print(clave, ' - > ',valor)
C - > 1972
python - > 1991
Java - > 1996
```

Cadenas y colecciones

Después de trabajar con listas podemos ver las cadenas como una lista de caracteres:

- Accedemos a cada carácter por índice *cadena[4]*.
- Podemos seleccionar una parte de una cadena *cadena[inicio:fin]*.

Veamos algunas funciones para trabajar con cadenas.

split: rompe una cadena en palabras

```
>>> frase = 'hola mundo'
>>> palabras = frase.split()
>>> print(palabras)
['hola', 'mundo']
```

Se le puede pasar como argumento el carácter separador, que por defecto es espacio.

join: genera cadena

Genera una cadena a partir de un separador y una lista

```
" ".join(['hola', 'mundo'])
```

Traducción de caracteres

Cuando trabajamos con algoritmos de encriptación sencillos podemos usar un diccionario para traducir el texto original en el texto codificado:

```
pt = dict()
pt['a'] = '7'
pt['e'] = ';'
pt['i'] = ' '
pt['o'] = '.'
pt['u'] = 'j'
# ....
```

A partir de un texto, generamos el texto codificado usando el diccionario que contiene la clave.

Conjuntos/Sets

También existen los **Conjuntos** (sets), que almacenan valores sin ordenar y que no pueden estar repetidos. Sólo se pueden guardar elementos inmutables (que no cambian) y que de los que se puede obtener un valor **hash** (algo similar a un id único).

La diferencia entre conjunto y lista es que en los conjuntos no se repiten elementos.

Además en los conjuntos no podemos acceder a los elementos individualmente.

Están pensados para trabajar las típicas operaciones entre conjuntos: uniones, intersecciones, diferencias,...

Los crearemos con los elementos entre llaves:

```
>>> s = {1, 2, 4}
```

Podemos añadir elementos con **add**

```
>>> print(s)
>>> s.add(5)
>>> print(s)
>>> s.add(5)
>>> print(s)
{1, 2, 4}
{1, 2, 4, 5}
{1, 2, 4, 5}
```

Y vemos que si intentamos añadir un elemento repetido, sólo aparecerá una vez.

Podemos hacer operaciones lógicas entre ellos, como por ejemplo esta operación **OR** en la que se muestra una unión de elementos:

```
>>> a = {1, 2, 3, 4}
>>> b = {3, 4, 5, 6}
>>> a | b
{1, 2, 3, 4, 5, 6}
```

Sería equivalente a una *unión*

```
>>> a.union(b)
{1, 2, 3, 4, 5, 6}
```

También podemos hacer una diferencia, que nos mostrará aquellos que están en uno y no el otro:

```
>>> a-b
{1, 2}
```

Zip

Al contrario de lo que pueda parecernos el nombre, la función **zip** no sirve para comprimir cosas sino para encajarlas. Quizás porque *zip* significa cremallera en inglés...

A partir de dos listas de elementos podemos generar una lista formada por pares de elementos, uno de cada lista

```
países = ['Francia', 'Alemania', 'España']
capitales = ['París', 'Berlín', 'Madrid']
```

```
for país, capital in zip(países, capitales):
    print(f'{país} capital {capital}')
```

También podemos generar un diccionario

```
diccionario = dict(zip(países, capitales))
print(diccionario)
```

Ejercicios colecciones

Vamos a ver algunos ejercicios para practicar lo aprendido con las colecciones.

Uniando 2 listas en un diccionario

Creemos un diccionario a partir de 2 listas: una contendrá las claves y otra los valores:

```
keys = ['Ten', 'Twenty', 'Thirty']
values = [10, 20, 30]

diccionario = {}
contador = 0
for key in keys:
    diccionario[key] = values[contador]
    contador += 1

print(diccionario)
```

Mezclar 2 diccionarios en uno

Unimos 2 diccionarios en uno que contendrá todos los pares:

```
dict1 = {'Ten': 10, 'Twenty': 20, 'Thirty': 30}
dict2 = {'Thirty': 30, 'Fourty': 40, 'Fifty': 50}

dict3 = dict1.copy() # Creamos un nuevo diccionario
dict3.update(dict2)
print(dict1)
```

Acceso a un elemento dentro de un diccionario

Se pueden crear estructuras anidadas dentro de un diccionario: el valor es un nuevo diccionario.

Acceder al valor del elemento 'history' en un diccionario que tiene esta estructura.

```
sampleDict = {
    "class":{
        "student":{
            "name":"Mike",
            "marks":{
                "physics":70,
                "history":80
            }
        }
    }
}

sampleDict["class"]["student"]["marks"]["history"]
```

Eliminar determinados pares de un diccionario

Vamos a ver cómo eliminar varios elementos de un diccionario:

```
diccionario = {
    "name": "Kelly",
    "age":25,
    "salary": 8000,
    "city": "New york"
}

clavesAQuitar = ["name", "salary"]
restantes = {}
for key in diccionario:
    if key not in clavesAQuitar:
        restantes[key] = diccionario[key]
print(restantes)
```